

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250220>

Reference group

Reference groups - quality assurance of education

Possible interface to give feedback on different matters related to the course.

Suggested 3 meetings, e.g. i) middle of February; ii) mid of March; and iii) end of April

The reference group:

- 1) Elisabella Hjelpdahl
- 2) Ana Rey Mata

Outline : Unsupervised Learning Techniques : Part 2 (code [samples](#))

- Definitions / Motivation
- Clustering
 - K-Means
 - Limits of K-Means
 - Using Clustering for Image Segmentation
 - Using Clustering for Preprocessing
 - Using Clustering for Semi-Supervised Learning
 - DBSCAN
 - Other Clustering Algorithms
- Gaussian Mixtures
 - Anomaly Detection Using Gaussian Mixtures
 - Selecting Number of Clusters
 - Bayesian Gaussian Mixture Models
 - Other Algorithms for Anomaly and Novelty Detection
- Examples

—

Source: Aurelien Geron, "Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems", 2019
([Available online](#)) / **Chapter 9**

Motivation

No labels. Collecting and labelling can easily become an expensive and time consuming task.

How to accommodate changes? “Re-labeling”?

Can the unlabeled data be explored?

Yes, so, unsupervised learning is the exploration of unlabeled data.

Dimensionality reduction is one example of unsupervised learning task.



Definitions

Clustering: group *similar* instances together into **clusters**. Great tool for data analysis, customer segmentation, recommender systems, search engines, image segmentation, semi-supervised learning, dimensionality reduction, and more.

Anomaly detection: learn what “normal” data looks like, and use this to **detect abnormal instances**, such as defective items on a production line or a new trend in a time series.

Density estimation: estimate the *probability density function* (PDF) of the random process that generated the dataset. This is commonly used for anomaly detection: instances located in very low-density regions are likely to be anomalies. It is also useful for data analysis and visualization.

K-Means : Algorithm

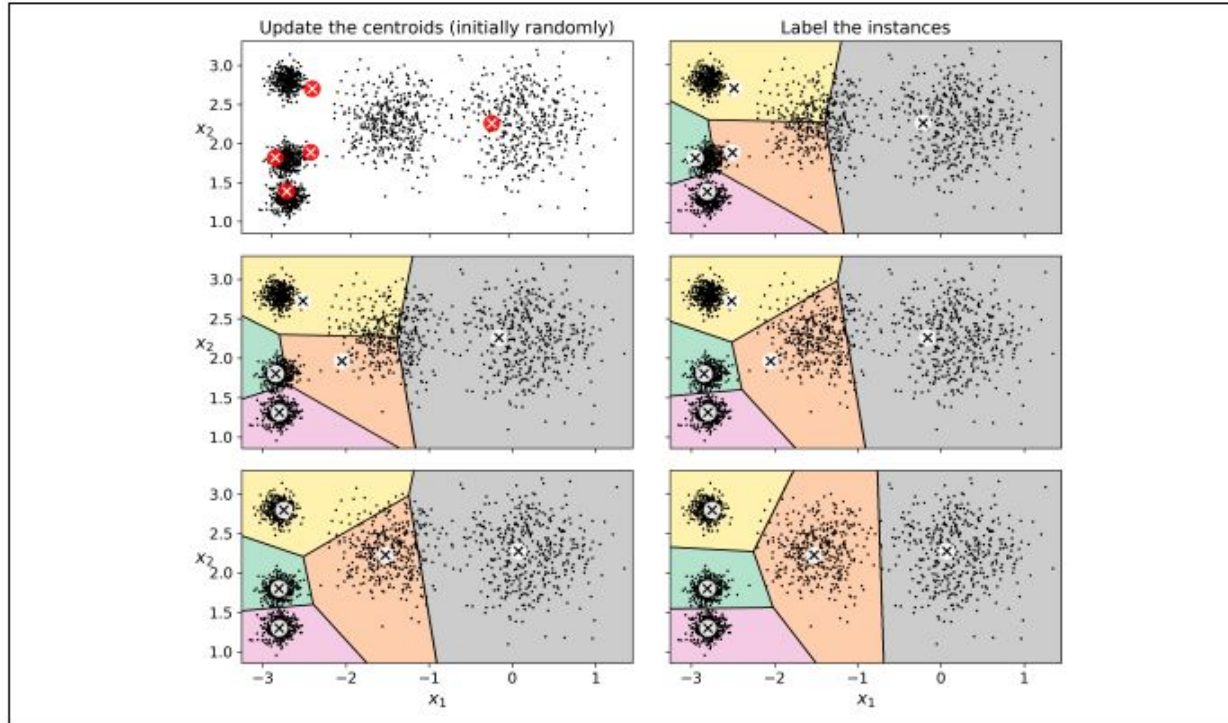


Figure 9-4. The K-Means algorithm

Using Clustering for Image Segmentation (1)

Steps 74-78

Image segmentation: partitioning an image into multiple segments.

In **semantic segmentation**, all pixels that are part of the same object type get assigned to the same segment. (e.g. *pedestrians* segment for self-driving car).

In **instance segmentation**, all pixels that are part of the same individual object are assigned to the same segment. (e.g. each *pedestrian* as a different segment).

Semantic and instance segmentation with the **convolutional neural networks**, here we look now at **color segmentation**.

- assign pixels to the *same segment* if they have a *similar color*.

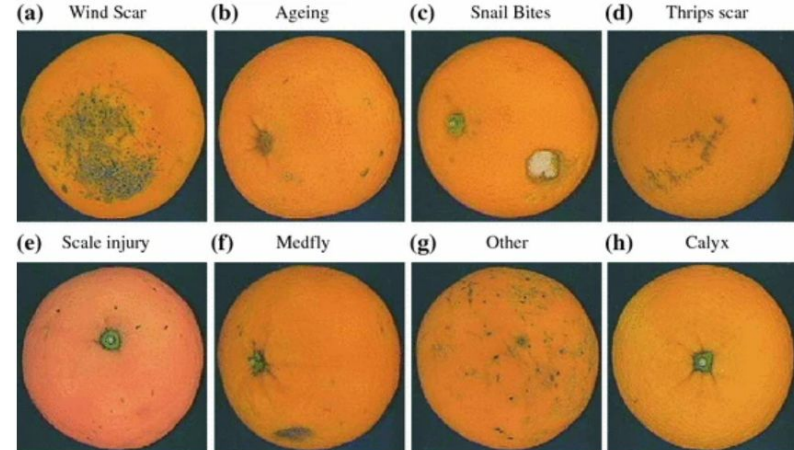
Using Clustering for Image Segmentation (2)

Examples:

Analyze satellite images to measure how much total forest area there is in a region, color segmentation may be just fine.

“Computer-based detection and classification of flaws in citrus fruits”.

“Rapid recognition method of red brick content in recycled brick-concrete aggregates and powder based on color segmentation”.



Example of different types of peel defects

Using Clustering for Image Segmentation (3)

```
>>> from matplotlib.image import imread # you could also use `imageio.imread()`  
>>> image = imread(os.path.join("images", "clustering", "ladybug.png"))  
>>> image.shape  
(533, 800, 3)
```

1st dimension - height, 2nd - width, and 3rd - number of color channels, in this case red, green and blue (RGB), each in [0..1].

```
X = image.reshape(-1, 3)  
kmeans = KMeans(n_clusters=8).fit(X)  
segmented_img = kmeans.cluster_centers_[kmeans.labels_]  
segmented_img = segmented_img.reshape(image.shape)
```

Using Clustering for Image Segmentation (4)



Figure 9-12. Image segmentation using K-Means with various numbers of color clusters

K-Means : Using for Preprocessing (1)

Steps 79-93

Can be an efficient approach to dimensionality reduction, in particular as a preprocessing step before a supervised learning algorithm.

MNIST-like dataset containing 1,797 grayscale 8×8 images representing digits 0 to 9.

```
from sklearn.datasets import load_digits
```

```
X_digits, y_digits = load_digits(return_X_y=True)
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X_digits, y_digits)
```

```
from sklearn.linear_model import LogisticRegression
```

```
log_reg = LogisticRegression(random_state=42)
```

```
log_reg.fit(X_train, y_train)
```

```
>>> log_reg.score(X_test, y_test)
0.9666666666666667
```

K-Means : Using for Preprocessing (2)

Although it is tempting to define the number of clusters to 10, since there are 10 different digits, it is unlikely to perform well, because there are several different ways to write each digit.

```
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ("kmeans", KMeans(n_clusters=50)),
    ("log_reg", LogisticRegression()),
])
pipeline.fit(X_train, y_train)
```

```
>>> pipeline.score(X_test, y_test)
0.9822222222222222
```

```
from sklearn.model_selection import GridSearchCV
```

```
param_grid = dict(kmeans__n_clusters=range(2, 100))
grid_clf = GridSearchCV(pipeline, param_grid, cv=3, verbose=2)
grid_clf.fit(X_train, y_train)
```

```
>>> grid_clf.best_params_
{'kmeans__n_clusters': 90}
>>> grid_clf.score(X_test, y_test)
0.9844444444444445
```

K-Means : Using Clustering for Semi-Supervised Learning (1) / [Steps 94-109](#)

Plenty of unlabeled instances and very few labeled instances.

→ Propagate known labels to the instances in the same clusters.

Example: 50 labeled instances from the digits dataset:

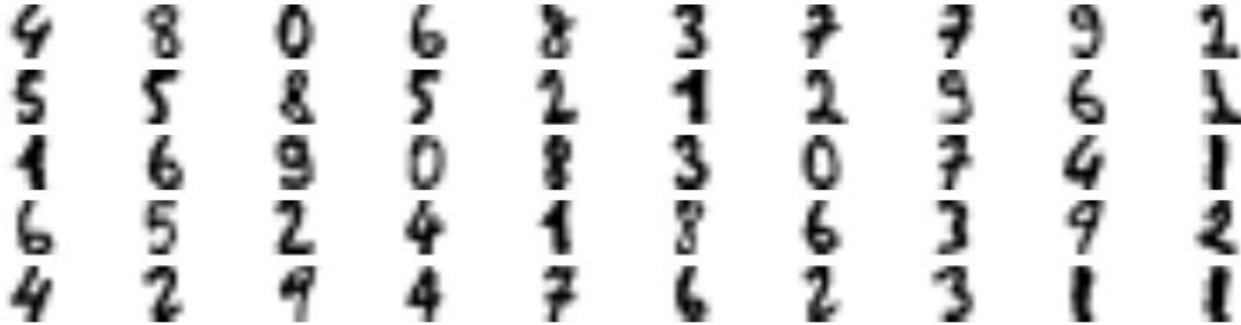


Figure 9-13. Fifty representative digit images (one per cluster)

K-Means : Using Clustering for Semi-Supervised Learning (2)

```
n_labeled = 50
log_reg = LogisticRegression()
log_reg.fit(X_train[:n_labeled], y_train[:n_labeled])
```

```
>>> log_reg.score(X_test, y_test)
0.8266666666666667
```

```
k = 50
kmeans = KMeans(n_clusters=k)
X_digits_dist = kmeans.fit_transform(X_train)
representative_digit_idx = np.argmin(X_digits_dist, axis=0)
X_representative_digits = X_train[representative_digit_idx]
y_representative_digits = np.array([4, 8, 0, 6, 8, 3, ..., 7, 6, 2, 3, 1, 1])
```

```
>>> log_reg = LogisticRegression()
>>> log_reg.fit(X_representative_digits, y_representative_digits)
>>> log_reg.score(X_test, y_test)
0.9244444444444444
```

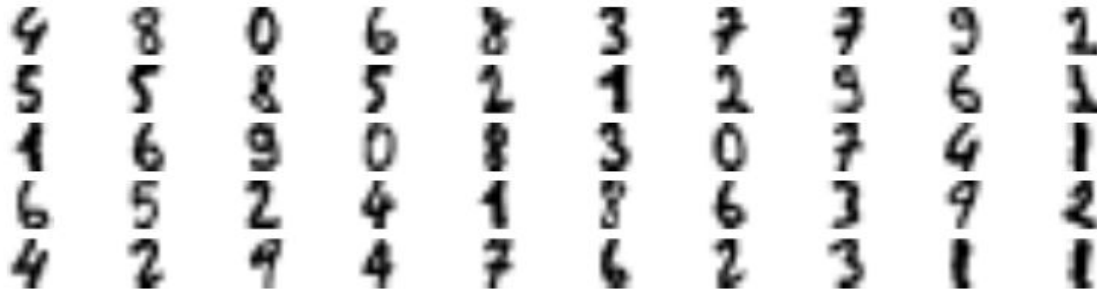


Figure 9-13. Fifty representative digit images (one per cluster)

Semi-Supervised Learning: Label propagation (1)

Propagating the labels to all the other instances in the same cluster:

```
y_train_propagated = np.empty(len(X_train), dtype=np.int32)
for i in range(k):
    y_train_propagated[kmeans.labels_==i] = y_representative_digits[i]
```

```
>>> log_reg = LogisticRegression()
>>> log_reg.fit(X_train, y_train_propagated)
>>> log_reg.score(X_test, y_test)
0.9288888888888889
```

Semi-Supervised Learning: Label propagation (2)

So, there was not an impressive improvement: 0,92(4) ~ 0,92(8)

What if to focus first on the instances classified with more confidence? E.g. the 20% of the instances that are closest to the centroids:

`percentile_closest = 20`

```
X_cluster_dist = X_digits_dist[np.arange(len(X_train)), kmeans.labels_]
for i in range(k):
    in_cluster = (kmeans.labels_ == i)
    cluster_dist = X_cluster_dist[in_cluster]
    cutoff_distance = np.percentile(cluster_dist, percentile_closest)
    above_cutoff = (X_cluster_dist > cutoff_distance)
    X_cluster_dist[in_cluster & above_cutoff] = -1
```

```
partially_propagated = (X_cluster_dist != -1)
X_train_partially_propagated = X_train[partially_propagated]
y_train_partially_propagated = y_train[partially_propagated]
```

With just 50 labeled instances
(only 5 examples per class on average!)
→ 94.2% performance

```
>>> log_reg = LogisticRegression()
>>> log_reg.fit(X_train_partially_propagated, y_train_partially_propagated)
>>> log_reg.score(X_test, y_test)
0.9422222222222222
```


Semi-Supervised Learning: Label propagation (3)

So, 94.2% performance, pretty close to the performance of logistic regression on the fully labeled digits dataset (which was 96.7%).

This is because the propagated labels are actually pretty good, their accuracy is very close to 99%:

```
>>> np.mean(y_train_partially_propagated == y_train[partially_propagated])  
0.9896907216494846
```

Also, active learning

Further possible improvements → **Active learning**: a human expert interacts with the learning algorithm, providing labels when the algorithm needs them.

Different strategies, most common **uncertainty sampling**:

- The model is trained on the labeled instances gathered so far, and this model is used to make predictions on all the unlabeled instances.
- The instances for which the model is most uncertain (i.e., when its estimated probability is lowest) must be labeled by the expert.
- Then you just iterate this process again and again, until the performance improvement stops being worth the labeling effort.

DBSCAN

Density-Based Spatial Clustering of Applications with Noise

DBSCAN: Density-Based Spatial Clustering of Applications with Noise

DBSCAN: a very different approach **based on local density estimation**. This approach allows the algorithm to **identify clusters of arbitrary shapes**.

Defining clusters as **continuous regions of high density**.

Works well if all the clusters are dense enough, and they are well separated by low-density regions.

DBSCAN: Defining clusters as continuous regions of high density

- For each instance, the algorithm counts how many instances are located within a small distance ϵ (epsilon) from it. This region is called the instance's ϵ - neighborhood.
- If an instance has at least `min_samples` instances in its ϵ -neighborhood (including itself), then it is considered a core instance. In other words, core instances are those that are located in dense regions.
- All instances in the neighborhood of a core instance belong to the same cluster. This may include other core instances, therefore a long sequence of neighboring core instances forms a single cluster.
- Any instance that is not a core instance and does not have one in its neighborhood is considered an anomaly.

DBSCAN (1) Steps 110-139

```
from sklearn.cluster import DBSCAN  
from sklearn.datasets import make_moons
```

```
X, y = make_moons(n_samples=1000, noise=0.05)  
dbscan = DBSCAN(eps=0.05, min_samples=5)  
dbscan.fit(X)
```

For -1 - means that the instances are considered as anomalies by the algorithm:

```
>>> dbscan.labels_  
array([ 0,  2, -1, -1,  1,  0,  0,  0, ...,  3,  2,  3,  3,  4,  2,  6,  3])
```


DBSCAN (2)

core_sample_indices_ : The indices of the core instances.

components_ : The core instances.

```
>>> len(dbscan.core_sample_indices_)
808
>>> dbscan.core_sample_indices_
array([ 0,  4,  5,  6,  7,  8, 10, 11, ..., 992, 993, 995, 997, 998, 999])
>>> dbscan.components_
array([[ -0.02137124,  0.40618608],
       [ -0.84192557,  0.53058695],
       ...,
       [ -0.94355873,  0.3278936 ],
       [  0.79419406,  0.60777171]])
```

DBSCAN (3)

$\epsilon=0.05$: a lot of anomalies, plus 7 different clusters. So, increasing ϵ , e.g. $\epsilon=0.2$.

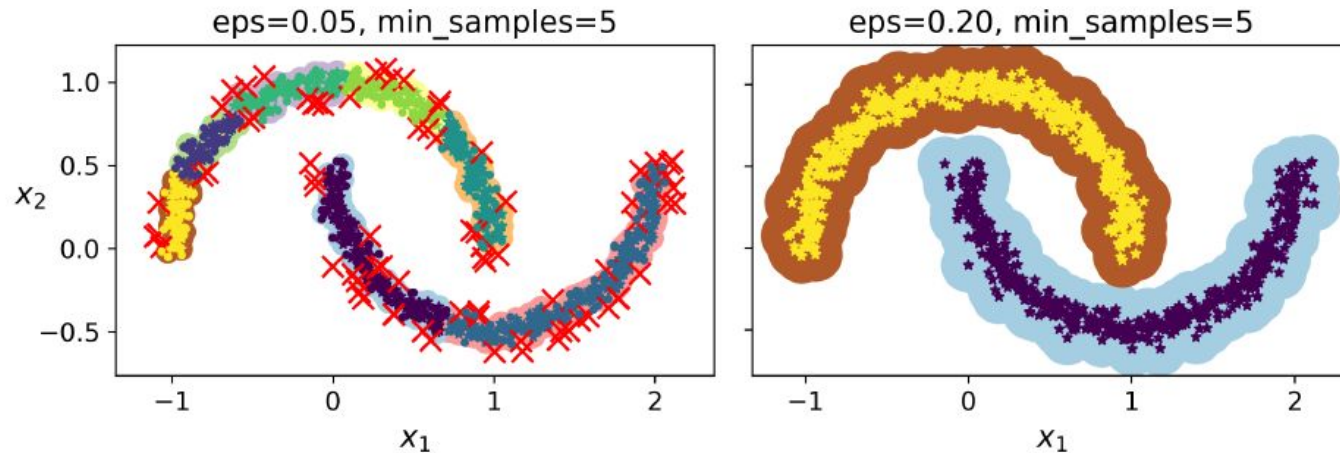


Figure 9-14. DBSCAN clustering using two different neighborhood radiuses

DBSCAN + KNeighborsClassifier

DBSCAN class does not have a predict() method.

There is a fit_predict() method. So, it cannot predict which cluster a new instance belongs to.

Several classification algorithms could make sense here, and it is easy enough to train one, for example a KNeighborsClassifier:

```
from sklearn.neighbors import KNeighborsClassifier
```

```
knn = KNeighborsClassifier(n_neighbors=50)  
knn.fit(dbscan.components_, dbscan.labels_[dbscan.core_sample_indices_])
```

DBSCAN + KNeighborsClassifier.predict(X_new) / predict_proba(X_new)

```
>>> X_new = np.array([[ -0.5,  0], [0,  0.5], [1, -0.1], [2,  1]])
>>> knn.predict(X_new)
array([1, 0, 1, 0])
>>> knn.predict_proba(X_new)
array([[0.18, 0.82],
       [1.   , 0.   ],
       [0.12, 0.88],
       [1.   , 0.   ]])
```

DBSCAN + KNeighborsClassifier / decision boundary

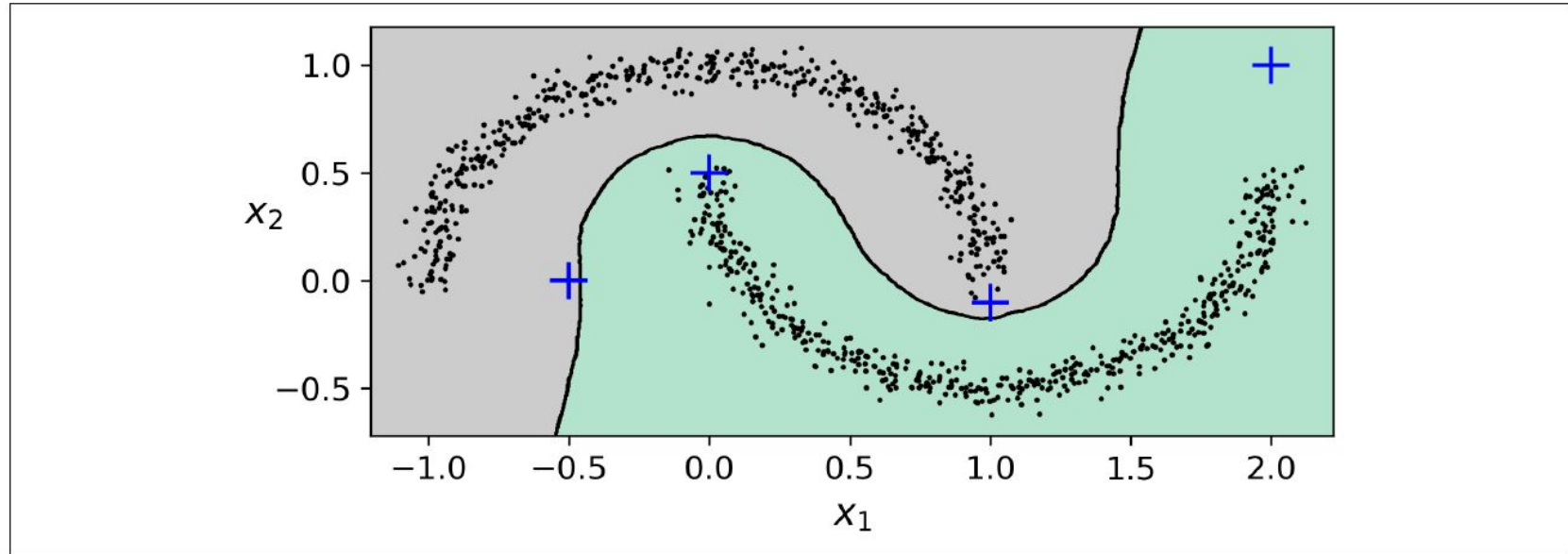


Figure 9-15. *cluster_classification_diagram*

DBSCAN + KNeighborsClassifier / defining anomalies through specifying a maximum distance (e.g. **y_dist**):

```
>>> y_dist, y_pred_idx = knn.kneighbors(X_new, n_neighbors=1)
>>> y_pred = dbscan.labels_[dbscan.core_sample_indices_][y_pred_idx]
>>> y_pred[y_dist > 0.2] = -1
>>> y_pred.ravel()
array([-1,  0,  1, -1])
```

DBSCAN : Overall

- Very simple yet powerful algorithm, capable of identifying any number of clusters, of any shape, it is robust to outliers, and it has just two hyperparameters (**eps** and **min_samples**).
- If the density varies significantly across the clusters: can be impossible for it to capture all the clusters properly.
- Computational complexity is roughly $O(m \log m)$, so pretty close to linear with regards to the number of instances.
- Scikit-Learn's implementation can require up to $O(m^2)$ memory if eps is large.

Other Clustering Algorithms

AgglomerativeClustering, Birch, MeanShift, AffinityPropagation,
SpectralClustering, ...

Gaussian Mixtures

Steps 140-172

Gaussian mixtures (1)

Gaussian mixture model (GMM) is a probabilistic model that assumes that the instances were generated from a mixture of several Gaussian distributions whose parameters are unknown.

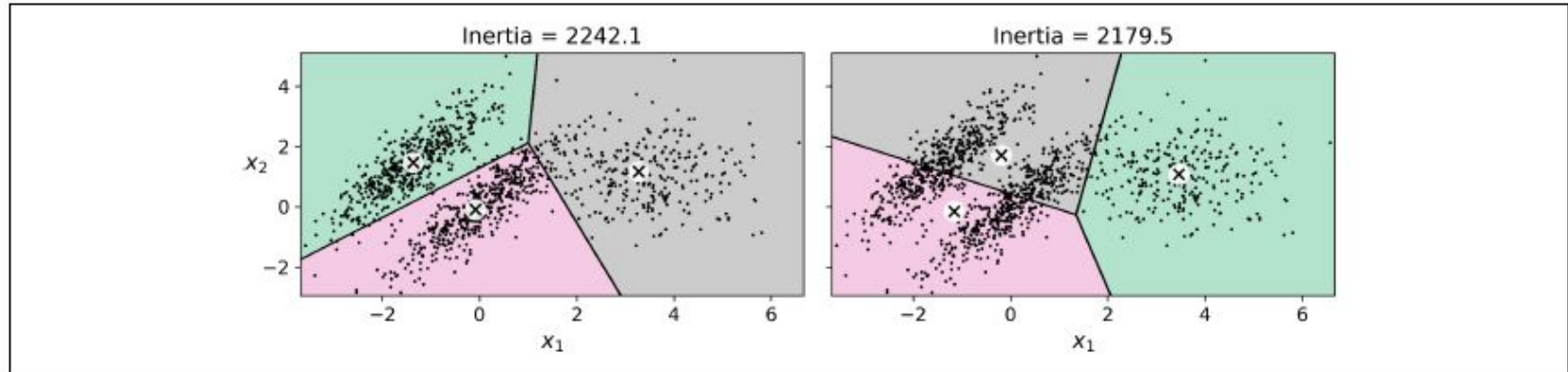


Figure 9-11. K-Means fails to cluster these ellipsoidal blobs properly

Gaussian mixtures (2)

All the instances generated from a single Gaussian distribution form a cluster that **typically looks like an ellipsoid**. Each **cluster** can have a **different ellipsoidal shape, size, density and orientation**.

It was generated from one of the Gaussian distributions, but you are not told which one, and you do not know what the parameters of these distributions are.

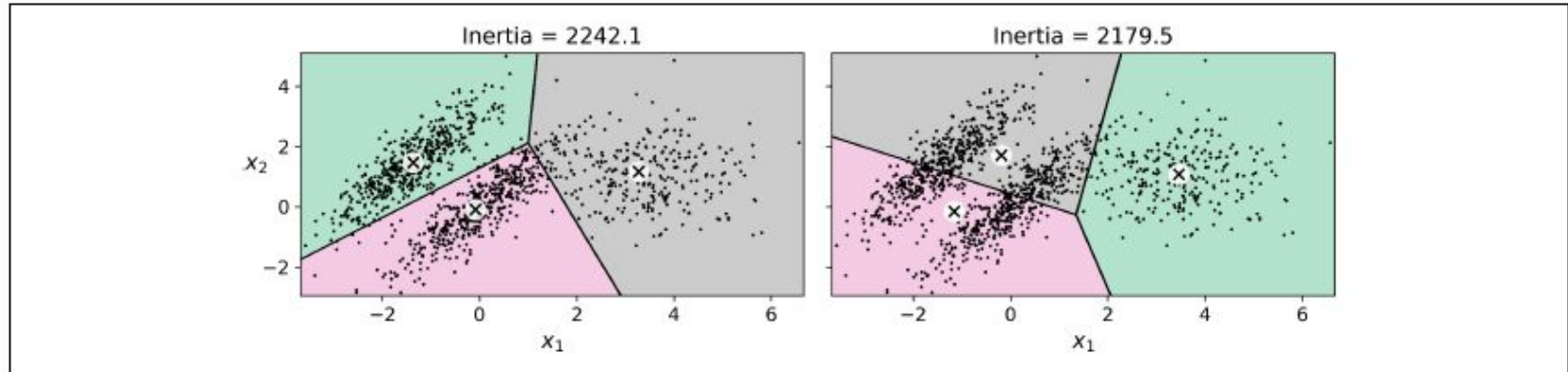


Figure 9-11. K-Means fails to cluster these ellipsoidal blobs properly

Gaussian mixtures (3)

Several GMM variants. For the simplest variant, implemented in the GaussianMixture class, one must know in advance the number k of Gaussian distributions.

$\mathbf{X}$ is assumed to have been generated through the following probabilistic process:

- For each instance, a cluster is picked randomly among k clusters. The probability of choosing the j^{th} cluster is defined by the cluster's weight $\phi^{(j)}$. The index of the cluster chosen for the i^{th} instance is noted $z^{(i)}$.
- If $z^{(i)}=j$, meaning the i^{th} instance has been assigned to the j^{th} cluster, the location $x^{(i)}$ of this instance is sampled randomly from the Gaussian distribution with mean $\mu^{(j)}$ and covariance matrix $\Sigma^{(j)}$. This is noted $x^{(i)} \sim N(\mu^{(j)}, \Sigma^{(j)})$.

Gaussian mixtures : Graphical model (plate notation)

- The circles represent random variables.
- The squares represent fixed values (i.e., parameters of the model).
- The large rectangles are called *plates*: they indicate that their content is repeated several times.

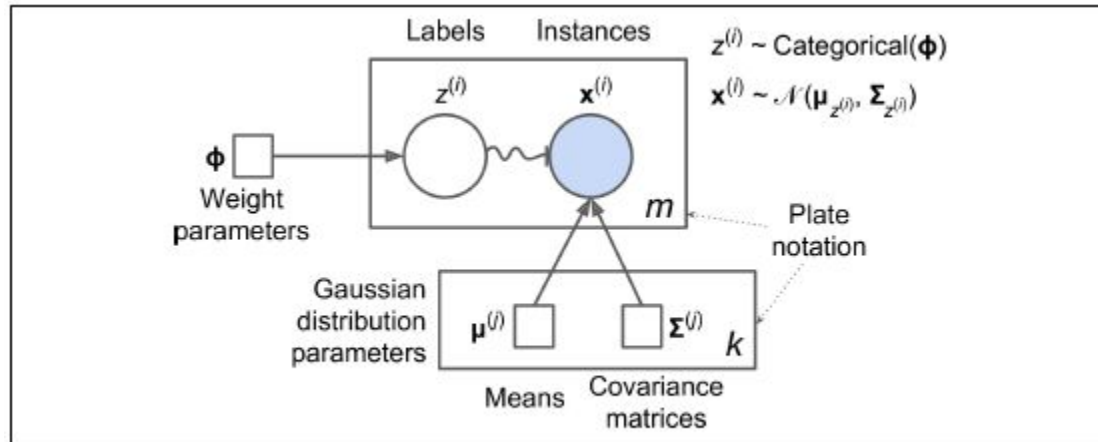


Figure 9-16. Gaussian mixture model

Gaussian mixtures : Graphical model (2)

- The number indicated at the bottom right hand side of each plate indicates how many times its content is repeated, so there are m random variables $z^{(i)}$ (from $z^{(1)}$ to $z^{(m)}$) and m random variables $\mathbf{x}^{(i)}$, and k means $\mu^{(j)}$ and k covariance matrices $\Sigma^{(j)}$, but just one weight vector Φ (containing all the weights $\phi^{(1)}$ to $\phi^{(k)}$).

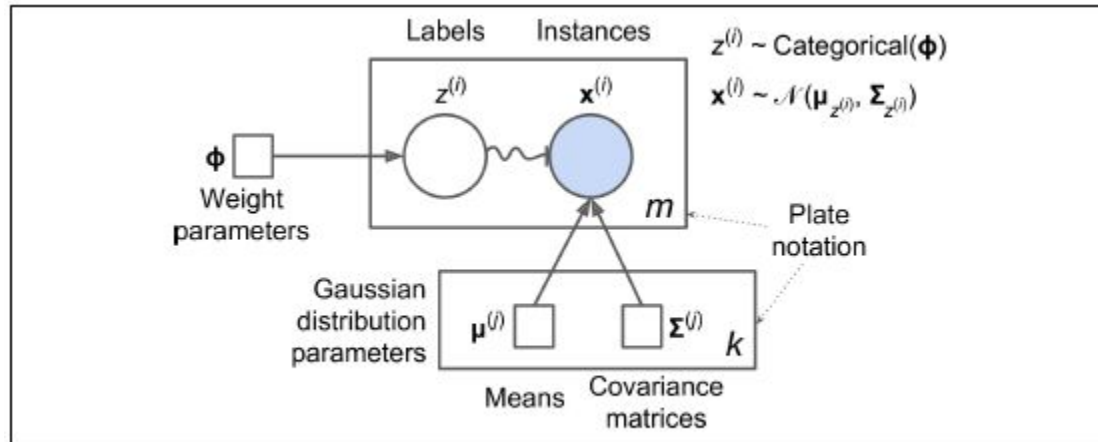


Figure 9-16. Gaussian mixture model

Gaussian mixtures : Graphical model (3)

- Each variable $z^{(i)}$ is drawn from the *categorical distribution* with weights ϕ . Each variable $\mathbf{x}^{(i)}$ is drawn from the normal distribution with the mean and covariance matrix defined by its cluster $z^{(i)}$.

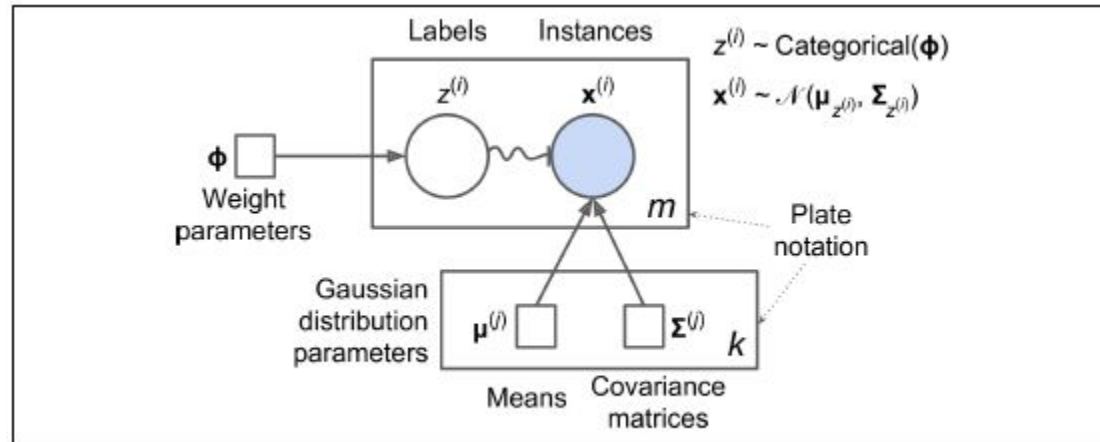


Figure 9-16. Gaussian mixture model

Gaussian mixtures : Graphical model (4)

- The solid arrows represent conditional dependencies. For example, the probability distribution for each random variable $z^{(i)}$ depends on the weight vector ϕ . Note that when an arrow crosses a plate boundary, it means that it applies to all the repetitions of that plate, so for example the weight vector ϕ conditions the probability distributions of all the random variables $\mathbf{x}^{(1)}$ to $\mathbf{x}^{(m)}$.

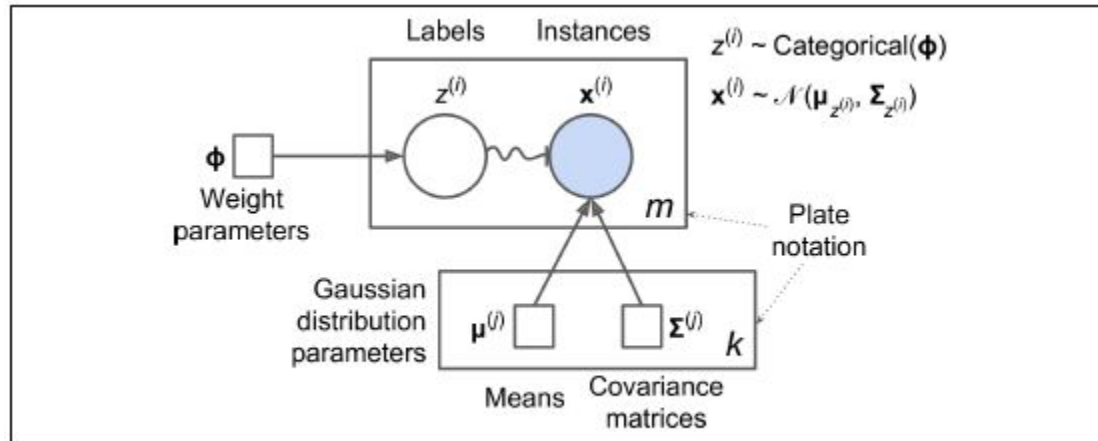


Figure 9-16. Gaussian mixture model

Gaussian mixtures : Graphical model (5)

- The squiggly arrow from $z^{(i)}$ to $\mathbf{x}^{(i)}$ represents a switch: depending on the value of $z^{(i)}$, the instance $\mathbf{x}^{(i)}$ will be sampled from a different Gaussian distribution. For example, if $z^{(i)}=j$, then $\mathbf{x}^{(i)} \sim N(\mu^{(j)}, \Sigma^{(j)})$.

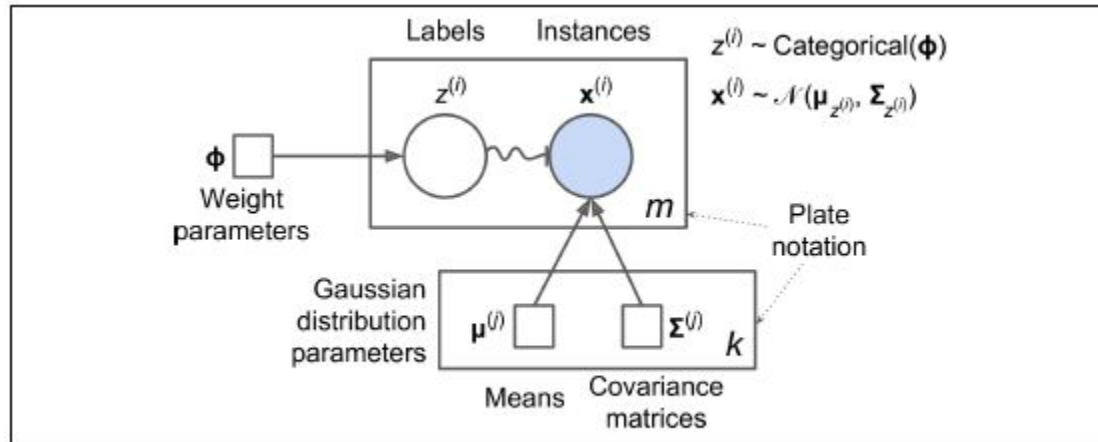


Figure 9-16. Gaussian mixture model

Gaussian mixtures : Graphical model (6)

- Shaded nodes indicate that the value is known, so in this case only the random variables $\mathbf{x}^{(i)}$ have known values: they are called *observed variables*. The unknown random variables $z^{(i)}$ are called latent variables.

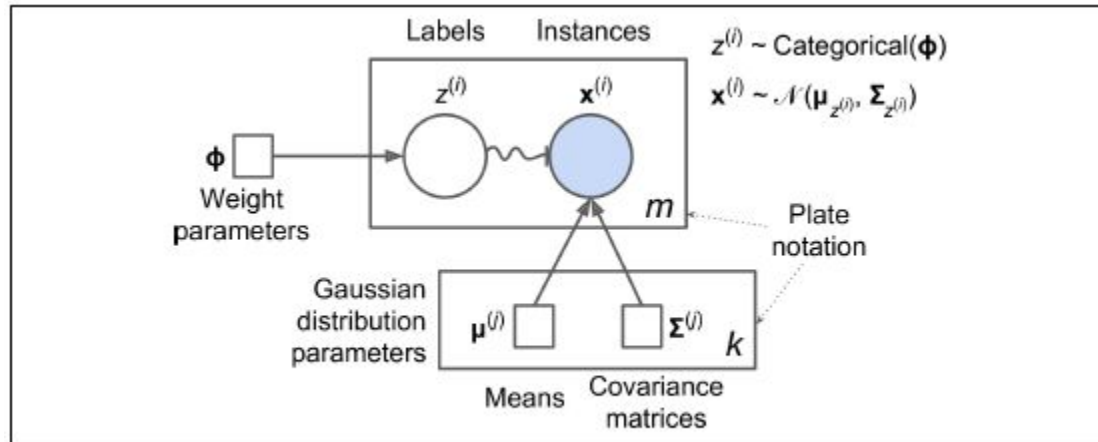


Figure 9-16. Gaussian mixture model

Gaussian mixtures : estimating the weights ϕ and the distribution parameters $\mu^{(1)}$ to $\mu^{(k)}$ and $\Sigma^{(1)}$ to $\Sigma^{(k)}$

$\mu^{(1)}$ to $\mu^{(k)}$: cluster centers,

$\Sigma^{(1)}$ to $\Sigma^{(k)}$: size, shape and orientation,

$\phi^{(1)}$ to $\phi^{(k)}$: cluster weights.

```
>>> gm.weights_  
array([0.20965228, 0.4000662 , 0.39028152])  
>>> gm.means_  
array([[ 3.39909717,  1.05933727],  
       [-1.40763984,  1.42710194],  
       [ 0.05135313,  0.07524095]])  
>>> gm.covariances_  
array([[[ 1.14807234, -0.03270354],  
        [-0.03270354,  0.95496237]],  
       [[ 0.63478101,  0.72969804],  
        [ 0.72969804,  1.1609872 ]],  
       [[ 0.68809572,  0.79608475],  
        [ 0.79608475,  1.21234145]]])
```

Gaussian mixtures : converged? → predictions

```
>>> gm.converged_
True
>>> gm.n_iter_
3

>>> X_new, y_new = gm.sample(6)
>>> X_new
array([[ 2.95400315,  2.63680992],
       [-1.16654575,  1.62792705],
       [-1.39477712, -1.48511338],
       [ 0.27221525,  0.690366  ],
       [ 0.54095936,  0.48591934],
       [ 0.38064009, -0.56240465]])

>>> y_new
array([0, 1, 2, 2, 2, 2])

>>> gm.predict(X)
array([2, 2, 1, ..., 0, 0, 0])
>>> gm.predict_proba(X)
array([[2.32389467e-02, 6.77397850e-07, 9.76760376e-01],
       [1.64685609e-02, 6.75361303e-04, 9.82856078e-01],
       [2.01535333e-06, 9.99923053e-01, 7.49319577e-05],
       ...,
       [9.99999571e-01, 2.13946075e-26, 4.28788333e-07],
       [1.00000000e+00, 1.46454409e-41, 5.12459171e-16],
       [1.00000000e+00, 8.02006365e-41, 2.27626238e-15]])

>>> gm.score_samples(X)
array([-2.60782346, -3.57106041, -3.33003479, ..., -3.51352783,
       -4.39802535, -3.80743859])
```

Gaussian mixtures : Means, decision boundaries (dashed lines), and density contours

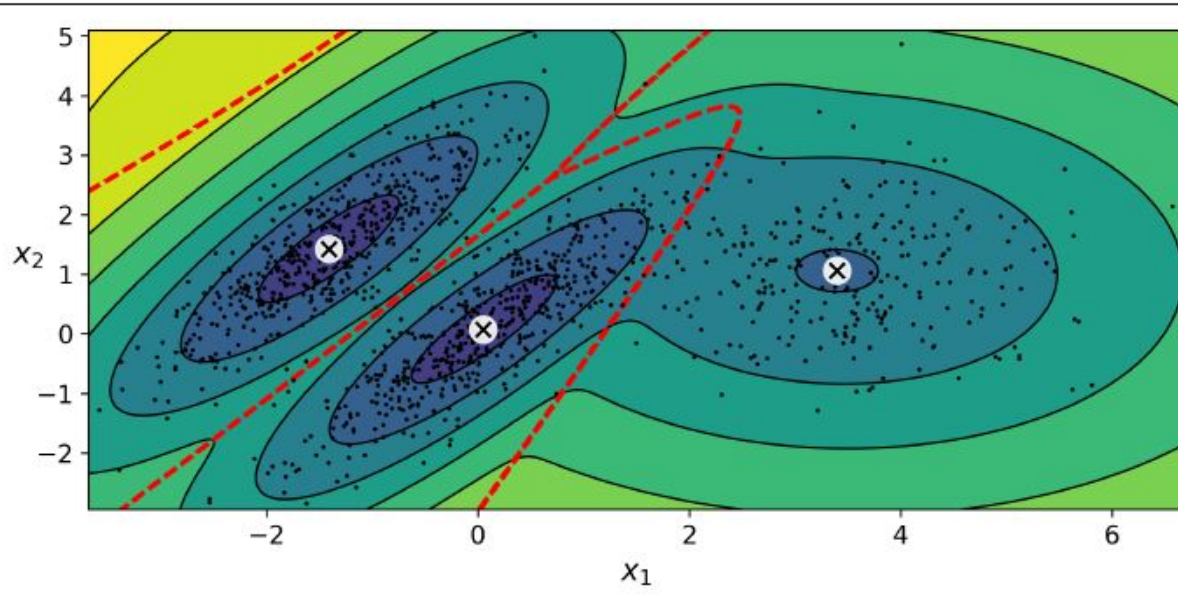


Figure 9-17. Cluster means, decision boundaries and density contours of a trained Gaussian mixture model

Computational complexity

Training a GaussianMixture model depends on the number of instances m , the number of dimensions n , the number of clusters k , and the constraints on the covariance matrices.

If **covariance_type** is "spherical" or "diag", it is $O(kmn)$, assuming the data has a clustering structure. If covariance_type is "tied" or "full", it is $O(kmn^2 + kn^3)$, so it will not scale to large numbers of features.

Anomaly detection

Define a density threshold to use
(e.g. 4%):

```
densities = gm.score_samples(X)
density_threshold = np.percentile(densities, 4)
anomalies = X[densities < density_threshold]
```

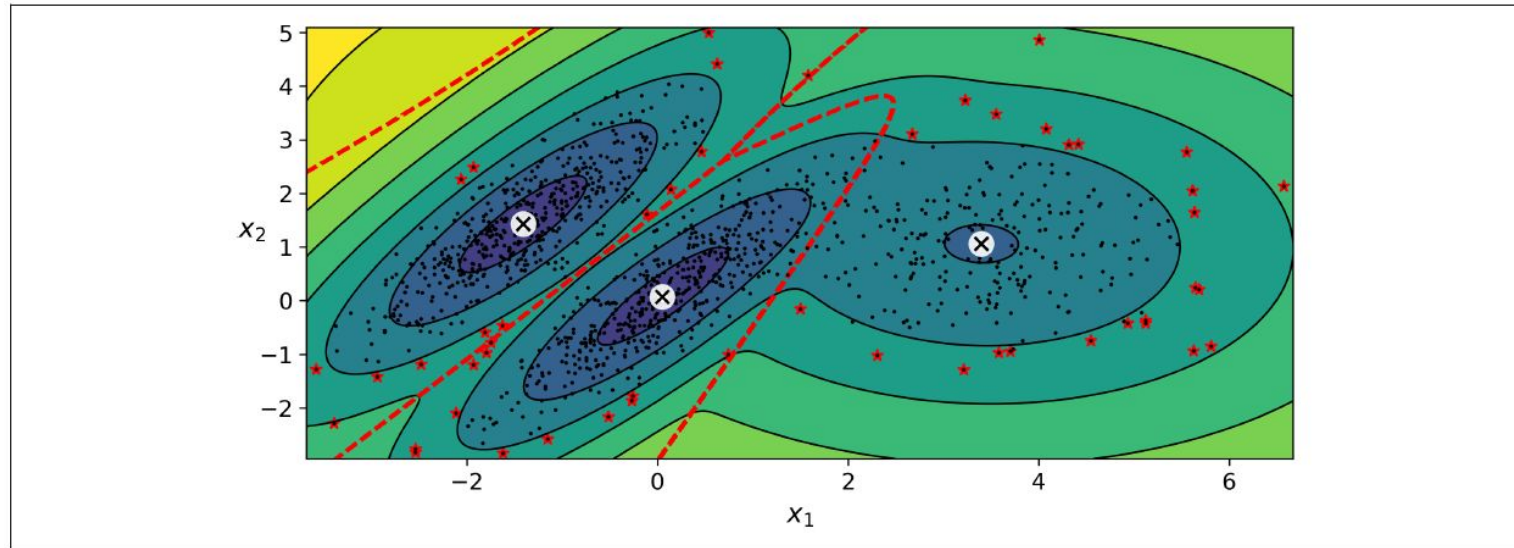


Figure 9-19. Anomaly detection using a Gaussian mixture model

Selecting Number of Clusters

Minimize minimizes a *theoretical information criterion* such as the *Bayesian information criterion* (BIC) or the *Akaike information criterion* (AIC)

Equation 9-1. Bayesian information criterion (BIC) and Akaike information criterion (AIC)

$$BIC = \log(m)p - 2 \log(\hat{L})$$

$$AIC = 2p - 2 \log(\hat{L})$$

- m - the number of instances,
- p - the number of parameters learned by the model,
- $\hat{L}$ - the maximized value of the *likelihood function* of the model.

Selecting Number of Clusters : BIC and AIC

- Both the BIC and the AIC penalize models that have more parameters to learn (e.g., more clusters), and reward models that fit the data well.
- They often end up selecting the same model, but when they differ, the model selected by the BIC tends to be simpler (fewer parameters) than the one selected by the AIC, but it does not fit the data quite as well (this is especially true for larger datasets).

Likelihood function

How well the model fits the data?

```
>>> gm.bic(X)
8189.74345832983
>>> gm.aic(X)
8102.518178214792
```

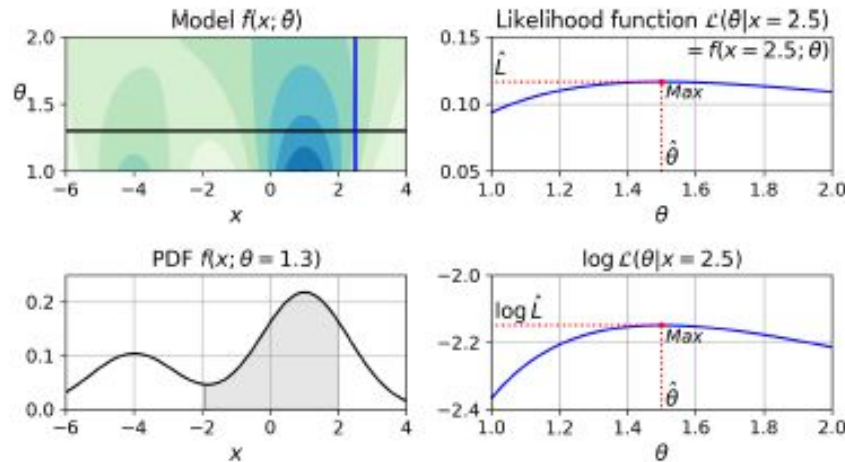


Figure 9-20. A model's parametric function (top left), and some derived functions: a PDF (lower left), a likelihood function (top right) and a log likelihood function (lower right)

Selecting Number of Clusters : BIC and AIC

$k=3$: the BIC and the AIC are lowest,
so it is most likely the best choice.

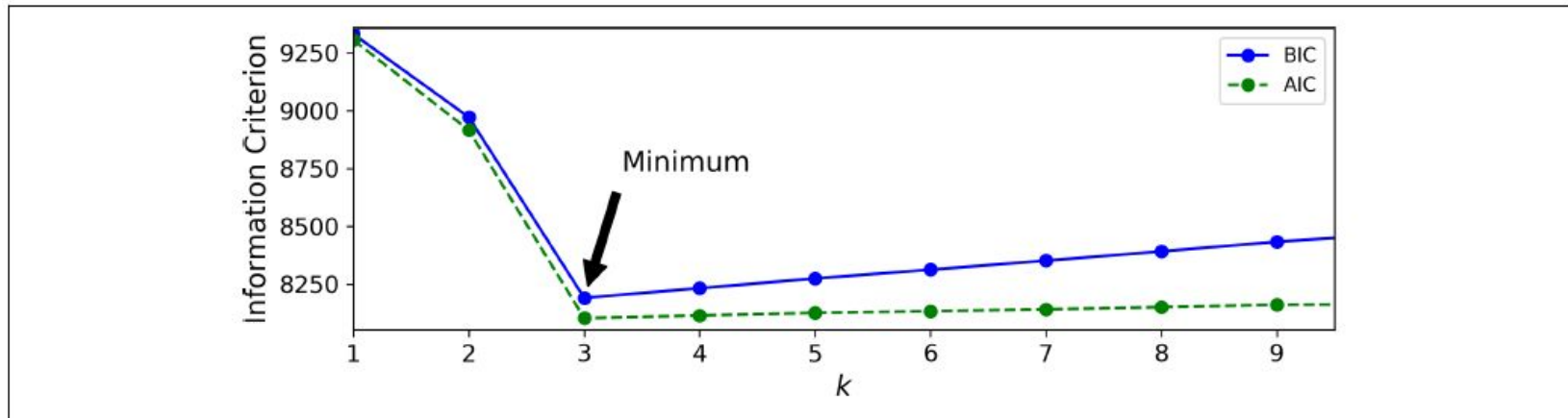


Figure 9-21. AIC and BIC for different numbers of clusters k

Bayesian Gaussian Mixture Models

Steps 173-186

Bayesian Gaussian Mixture Models (1)

BayesianGaussianMixture class gives weights equal (or close) to zero to unnecessary clusters. Set **n_components** to a “good value”.

```
>>> from sklearn.mixture import BayesianGaussianMixture
>>> bgm = BayesianGaussianMixture(n_components=10, n_init=10, random_state=42)
>>> bgm.fit(X)
>>> np.round(bgm.weights_, 2)
array([0.4 , 0.21, 0.4 , 0. , 0. , 0. , 0. , 0. , 0. , 0. ])
```

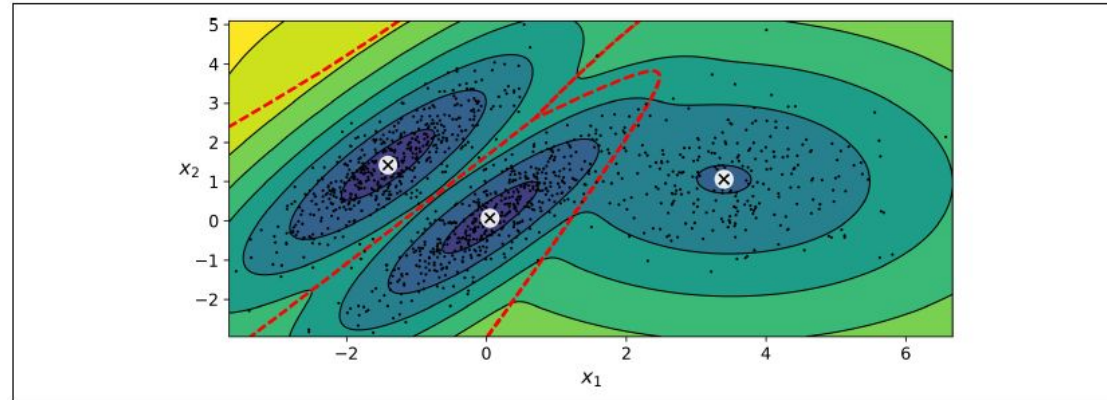


Figure 9-17. Cluster means, decision boundaries and density contours of a trained Gaussian mixture model

Bayesian Gaussian Mixture Models (2)

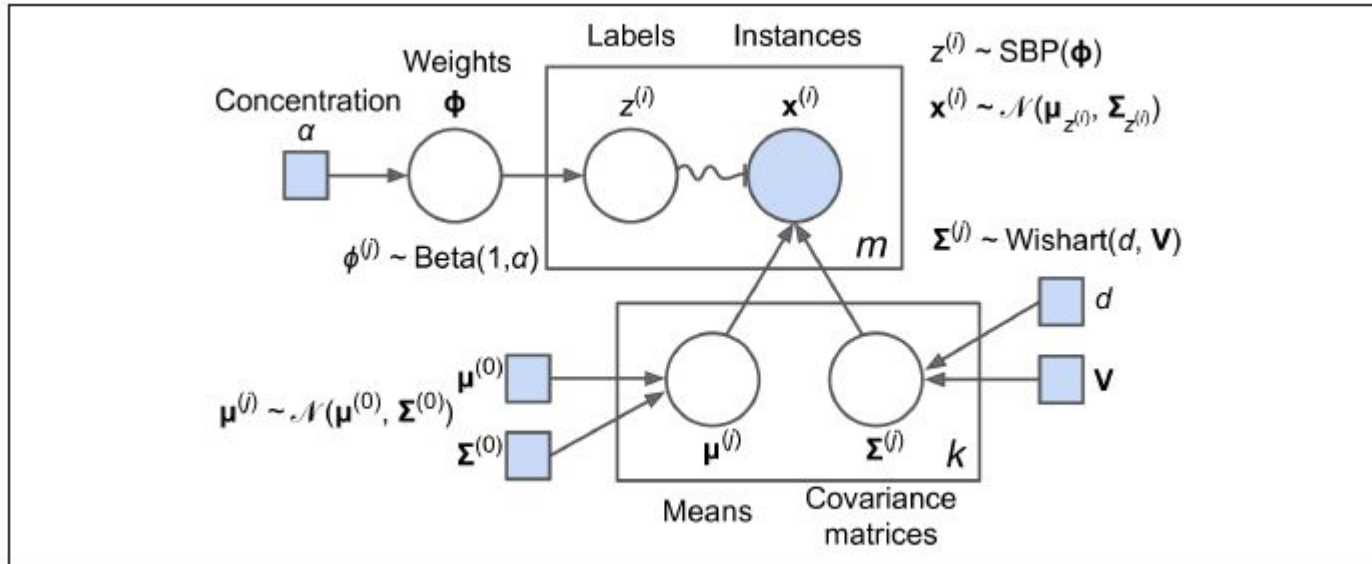


Figure 9-22. Bayesian Gaussian mixture model

Bayesian Gaussian Mixture Models (3)

Prior belief on few or many clusters.

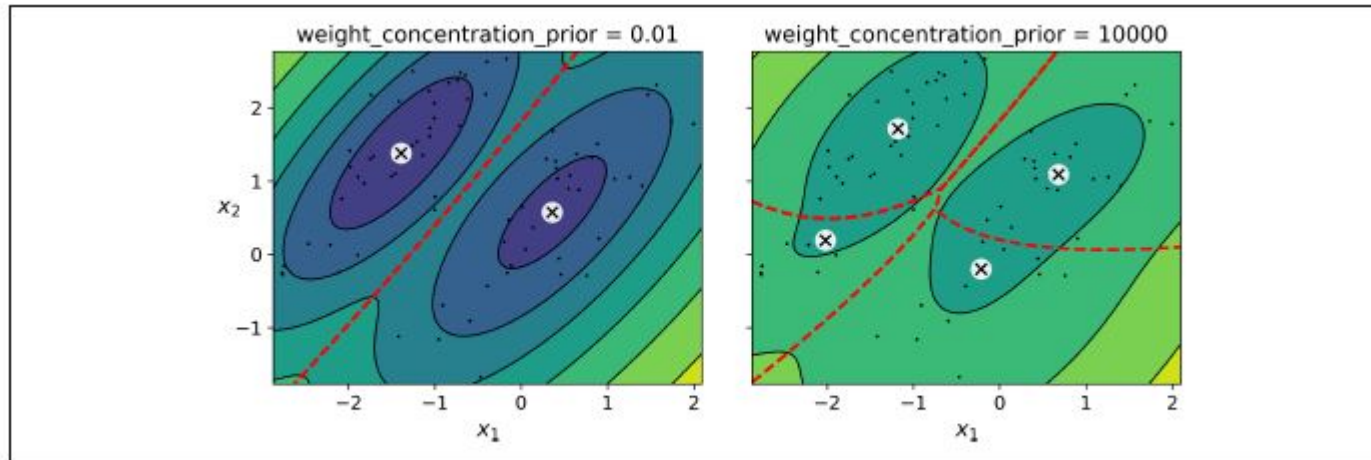


Figure 9-23. Using different concentration priors

Bayesian Gaussian Mixture Models (4)

Equation 9-2. Bayes' theorem

$$p(\mathbf{z}|\mathbf{X}) = \text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}} = \frac{p(\mathbf{X}|\mathbf{z}) p(\mathbf{z})}{p(\mathbf{X})}$$

To consider all possible combinations of cluster parameters and cluster assignments.

Equation 9-3. The evidence $p(\mathbf{X})$ is often intractable

$$p(\mathbf{X}) = \int p(\mathbf{X}|\mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

Bayesian Gaussian Mixture Models (5)

Addressing intractability via evidence lower bound (ELBO)

Equation 9-4. KL divergence from $q(z)$ to $p(z|X)$

$$\begin{aligned}
 D_{KL}(q \parallel p) &= \mathbb{E}_q \left[\log \frac{q(z)}{p(z | X)} \right] \\
 &= \mathbb{E}_q [\log q(z) - \log p(z | X)] \\
 &= \mathbb{E}_q \left[\log q(z) - \log \frac{p(z, X)}{p(X)} \right] \\
 &= \mathbb{E}_q [\log q(z) - \log p(z, X) + \log p(X)] \\
 &= \mathbb{E}_q [\log q(z)] - \mathbb{E}_q [\log p(z, X)] + \mathbb{E}_q [\log p(X)] \\
 &= \mathbb{E}_q [\log p(X)] - (\mathbb{E}_q [\log p(z, X)] - \mathbb{E}_q [\log q(z)]) \\
 &= \log p(X) - \text{ELBO} \\
 &\quad \text{where ELBO} = \mathbb{E}_q [\log p(z, X)] - \mathbb{E}_q [\log q(z)]
 \end{aligned}$$

Bayesian Gaussian Mixture Models (6)

Gaussian mixture models work great on clusters with ellipsoidal shapes. But for a dataset with different shapes, you may have bad surprises.

Searching for ellipsoids - 8 clusters instead of 2.

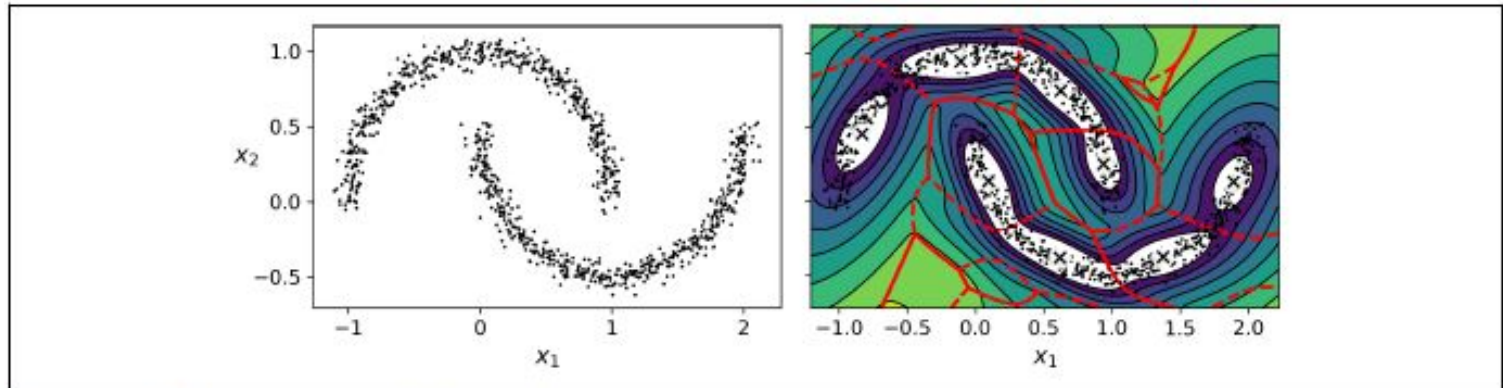


Figure 9-24. *moons_vs_bgm_diagram*

Other Algorithms for Anomaly and Novelty Detection

Other algorithms

Fast-MCD (minimum covariance determinant): EllipticEnvelope

Isolation forest: IsolationForest

Local outlier factor (LOF): LocalOutlierFactor

One-class SVM: SGDOneClassSVM (finding a small region that encompasses all the instances. If a new instance does not fall within this region, it is an anomaly.)

Takeaways

- K-Means for image segmentation, preprocessing, semi-supervised learning (labelling)
- Density-Based Spatial Clustering of Applications with Noise - robust solution for density does not vary significantly across the clusters.
- Gaussian mixture models for density estimation, clustering and anomaly detection.